GO DEEPER

# FUNZIONI



```
package main

import "fmt"

func add(x int, y int) int {

    return x + y

}

func main() {

    fmt.Println(add(42, 13))

}
```



## VALORI DI RITORNO MULTIPLI

```go
package main

import "fmt"

func divide(x int, y int) (int, int) {

    return x / y, x % y

}

func main() {

    fmt.Println(divide(7, 3)) // Stampa: 2 1

}
```



## VALORI DI RITORNO NOMINATI

```go
package main

import "fmt"

func divide(x int, y int) (div int, mod int) {

    div = x / y
    mod = x % y
    return // Restituisce div e mod

}

func main() {
    fmt.Println(divide(7, 3)) // Stampa: 2 1
}
```

## PASSAGGIO DI VALORE

- Go usa sempre il **passaggio per valore** (passaggio di copia).
- Una **copia del dato** viene passata alla funzione.
- **Modifiche interne non visibili** all'esterno.
- Usato per tipi base: `int`, `string`, `struct` piccole.

```go
func increment(x int) {
    x++ // Modifica solo la COPIA locale
}

func main() {
    a := 5
    increment(a)
    // 'a' non cambia.
}
```

## PASSAGGIO DI RIFERIMENTO (PUNTATORE)

- Viene passata la **copia dell'indirizzo di memoria** (*T).
- La funzione modifica il **valore originale** tramite l'indirizzo.
- Usato per: `struct` grandi (efficienza) o quando è necessaria la modifica.

```
func scale(v *int) {
    *v *= 10 // Dereferenzia e modifica l'ORIGINALE
}

func main() {
    a := 5
    scale(&a) // Passa l'indirizzo di 'a'
    // 'a' ora è 50
}
```



- **Slice, Map, Channel**: Tipi di riferimento. La loro *struttura* è copiata, ma il dato sottostante è condiviso

VALORI FUNZIONE

- Le funzioni possono essere passate come qualsiasi altro valore.

```go
// Funzione ipotenusa d'esempio
func hypot(x, y float64) float64 {
    return math.Sqrt(x*x + y*y)
}

// Questa funzione accetta una funzione come argomento
func compute(fn func(float64, float64) float64) float64 {
    return fn(3, 4)
}

func main() {
    // Passiamo la funzione hypot
    fmt.Println(compute(hypot)) // Stampa: 5

    // Passiamo la funzione math.Pow
    fmt.Println(compute(math.Pow)) // Stampa: 81 (cioè Pow(3, 4))
}
```

## CLOSURE

- Una funzione che fa riferimento a variabili dall'esterno del suo corpo.
- Ogni istanza di closure è legata alla **propria copia** delle variabili esterne (un'unica variabile `sum` per ogni chiamata ad `adder()`).

Non ne capisco l'utilità



```go
package main

import "fmt"

// adder restituisce una closure
func adder() func(int) int {
    sum := 0 // Questa variabile è "catturata" dalla closure
    return func(x int) int {
        sum += x
        return sum
    }
}

func main() {
    // Creiamo due closure indipendenti, ognuna con la propria 'sum'
    pos, neg := adder(), adder()
    for i := 0; i < 10; i++ {
        fmt.Println(
            // pos e neg mantengono stati separati
            pos(i),
            neg(-2*i),
        )
    }
}
```

## GESTIONE DEGLI ERRORI

Go usa **valori di ritorno** espliciti per gli errori. Non usa `try...catch` (eccezioni)



## IL TIPO ERROR

- `error` è un'interfaccia predefinita in Go.
- Implementa il metodo `Error() string`.
- Le funzioni fallibili restituiscono `error` come **ultimo** valore.



PATTERN

Si verifica sempre l'errore subito dopo la chiamata.

```go
func Divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("divisione per zero")
    }
    return a / b, nil
}

func main() {
    result, err := Divide(10, 0)

    // Controlla subito se c'è un errore
    if err != nil {
        fmt.Println("Errore:", err)
        return
    }
    fmt.Println("Risultato:", result)
}
```

## LA TIPOLOGIA DELL'ERRORE

- Il **tipo** specifico (una `struct` o una variabile `error`) che implementa l'interfaccia `error`.
- Indica la **causa** o la **natura** del fallimento (es. `*os.PathError`, `ErrNotFound`).
- Permette un controllo logico specifico.
- Si controlla con `errors.Is` (uguaglianza) o `errors.As` (conversione).



```go
// Esempio: Controllo sul TIPO di errore
var ErrInvalidInput = errors.New("input non valido")

func Process(data string) error {
    if data == "" {
        // Restituisco il VALORE specifico ErrInvalidInput
        return ErrInvalidInput
    }
    return nil
}

func main() {
    err := Process("")

    // Controllo sul TIPO (errore specifico)
    if errors.Is(err, ErrInvalidInput) {
        fmt.Println("Errore logico: input mancante.")
    } else if err != nil {
        fmt.Println("Errore generico:", err)
    }
}
```

```go
type AuthError struct {
    User string
    Code int
    Msg  string
}

// Implementazione del metodo Error() per l'interfaccia error
func (e *AuthError) Error() string {
    return fmt.Sprintf("Auth fallita per %s: %s (Codice %d)", e.User, e.Msg,
e.Code)
}

func Authenticate(user string) error {
    // In caso di errore specifico
    return &AuthError{User: user, Code: 401, Msg: "Credenziali non valide"}
}
```

Posso decidere io di che tipo deve essere error

```go
func main() {
    err := Authenticate("ospite")

    var authErr *AuthError

    // errors.As tenta di assegnare l'errore a 'authErr' se il tipo corrisponde
    if errors.As(err, &authErr) {
        fmt.Println("Autenticazione Fallita:")
        fmt.Printf("- Utente: %s\n", authErr.User)
        fmt.Printf("- Codice: %d\n", authErr.Code)
    } else if err != nil {
        fmt.Println("Errore generico:", err)
    }
}
```

# METODI



## STRUCT: RIASSUNTO

Le `struct` sono collezioni tipizzate di campi.

```
type Vertex struct {
    X int
    Y int
}

// In qualche punto del codice
v := Vertex{1, 2}
// fmt.Println(v)
```



## METODI SU STRUCT

I metodi sono funzioni con un argomento *receiver* (ricevente) speciale.



## RECEIVER PER VALORE (COPIA)

```
type Vertex struct {
    X int
    Y int
}

// Receiver per Valore: opera su una COPIA della struct
func (v Vertex) Equal(other Vertex) bool {
    return v.X == other.X && v.Y == other.Y
}

// Uso nel codice
// vtx := Vertex{1, 2}
// if vtx.Equal(Vertex{2, 3}) { ... }
```



## RECEIVER PER PUNTATORE (MODIFICA)

```
type Vertex struct {
    X int
    Y int
}

// Receiver per Puntatore: opera sulla struct ORIGINALE
// e può modificarne i campi.
func (v *Vertex) Scale(factor int) {
    v.X *= factor
    v.Y *= factor
}

// Uso nel codice
// vtx := &Vertex{1, 2}
// vtx.Scale(10) // Ora vtx.X e vtx.Y sono moltiplicati per 10
```

## METODI SU TIPI PERSONALIZZATI

I metodi possono essere definiti su qualsiasi tipo (ad eccezione dei tipi puntatore o di interfaccia).



## TIPI DI BASE CON METODI

```go
// Tipi personalizzati basati su float64
type Latitude float64

// Metodo definito su Latitude
func (lat *Latitude) IsValid() bool {
    // *lat è il valore
    return lat != nil && *lat >= -90 && *lat <= 90
}

type Longitude float64

// Metodo definito su Longitude
func (lng *Longitude) IsValid() bool {
    return lng != nil && *lng >= -180 && *lng <= 180
}
```



## STRUCT CHE USANO TIPI CON METODI

I metodi consentono di riutilizzare la logica di validazione nei tipi più complessi

```
type Point2D struct {
    Latitude  Latitude
    Longitude Longitude
}

// Metodo definito su Point2D
func (p Point2D) IsValid() bool {
    // Chiama i metodi IsValid() definiti sui campi Latitude e Longitude
    return p.Latitude.IsValid() && p.Longitude.IsValid()
}

func (p Point2D) IsZero() bool {
    return p.Latitude == 0 && p.Longitude == 0
}
```

stessa firma

# INTERFACCE

## DEFINIZIONE DI INTERFACCIA

Un'interfaccia definisce un **insieme di firme di metodi**.

- Un tipo implementa un'interfaccia se implementa **tutti** i metodi definiti nell'interfaccia.
- L'implementazione è **implicita** (non è richiesta la parola chiave `implements`).



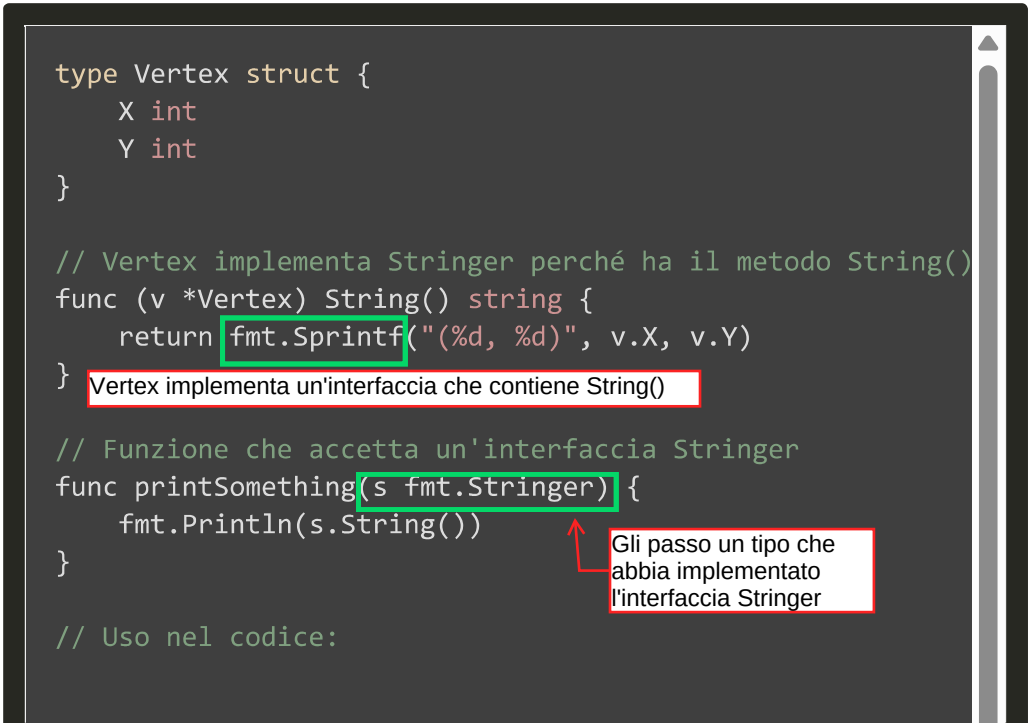
## INTERFACCIA STRINGER

Un esempio comune della libreria standard.

```go
// fmt/print.go:62
type Stringer interface {
    String() string
}
```

## STRUCT CHE IMPLEMENTA STRINGER

Qualsiasi tipo che ha il metodo `String() string` implementa implicitamente `Stringer`.



```
// v := &Vertex{10, 20}
// printSomething(v) // Output: (10, 20)
```

## L'INTERFACCIA WRITER

L'interfaccia `io.Writer` è fondamentale in Go per l'I/O (Input/Output).

```
// io/io.go:96
type Writer interface {
    Write(p []byte) (n int, err error)
}
```

### IMPLEMENTAZIONI COMUNI

Qualsiasi cosa che può accettare byte implementa `Writer`.

- `os.Stdout`: Lo standard output della console.
- `bytes.Buffer`: Un buffer di byte in memoria.
- `os.File`: Qualsiasi file su disco.
- `net.Conn`: Una connessione di rete (socket).

Le interfacce consentono il **polimorfismo** e il **disaccoppiamento**, permettendo a funzioni come `fmt.Fprintf` di lavorare con qualsiasi `Writer`.



## COMPOSIZIONE VS. EREDITARIETÀ

Go non ha l'ereditarietà di classe. Favorisce il riuso del codice tramite la **composizione**.

- Evita la complessità dell'ereditarietà multipla.
- Approccio: "Fa qualcosa" (Interfaccia) vs. "È un tipo di" (Classe).



## COMPOSIZIONE (EMBEDDING)

Si incorpora una `struct` all'interno di un'altra.

- Embedding: Inserire una `struct` **senza nome di campo**.
- Promozione: I metodi della struct interna sono **promossi** alla struct esterna.
- Simula il riutilizzo **senza** ereditarietà di tipo.



```go
type Logger struct {
    LogLevel string
}
func (l Logger) Log(msg string) {
    // Logica di logging...
}

// Server 'ha' un Logger (Composizione)
type Server struct {
    Logger // Embedding
    Host string
}

func main() {
    s := Server{Logger: Logger{LogLevel: "INFO"}}
    // Il metodo Log è ora accessibile direttamente da 's'
    s.Log("Avvio del server.")
}
```

s non ha inizializzato membro Host (stringa), allora Host sarà stringa vuota

